



Dynamic Programming



Definition

Dynamic programming is a method for solving complex problems by breaking them down into simpler subproblems and solving each subproblem only once, storing its solution to avoid redundant calculations. This approach is particularly useful for optimization problems where the solution involves making a sequence of decisions.



Fibonacci Sequence

It is a series of numbers where each number is the sum of the two preceding ones, usually starting with 0 and 1. The sequence begins: 0, 1, 1, 2, 3, 5, 8, 13, 21 ...

The formula to generate the Fibonacci sequence is:

$$F(n)=F(n-1)+F(n-2)$$



Fibonacci Sequence Code

```
int fib(int n) {  
    if(n == 0) return 0;  
    if(n == 1) return 1;  
    return fib(n-1) + fib(n-2);  
}
```



Fibonacci Using DP

```
int dp[25];
int fib(int n) {
    if(n == 0) return 0;
    if(n == 1) return 1;
    if(dp[n] != -1) {
        cout << "Returning from saved data for fib " << n << endl;
        return dp[n];
    }
    cout << "Calculating fibonacci " << n << endl;
    dp[n] = fib(n-1) + fib(n-2);

    return dp[n];
}
```



Dynamic Problem Approaches

- **Top-down approach (Memoization)**: In this approach, the problem is solved recursively, but with a mechanism to store the solutions of subproblems so that they can be reused. This prevents redundant recursive calls by checking if a solution to a subproblem has already been computed.
- **Bottom-up approach (Tabulation)**: In this approach, the problem is solved iteratively starting from the smallest subproblems and gradually building up to the larger ones. Solutions to subproblems are stored in a table and reused as needed to compute solutions to larger problems.



Fibonacci Number (Bottom Up)

```
int fib[25];  
fib[0] = 0;  
fib[1] = 1;  
cin >> fibNumber;  
for(int i = 2; i<=fibNumber; i++) {  
    fib[i] = fib[i-1] + fib[i-2];  
}  
cout << fib[fibNumber] << endl;
```



Rod Cutting

The rod cutting problem is a classic optimization problem often used to illustrate dynamic programming techniques. In this problem, we're given a rod of length n inches and a list of prices for rods of different lengths. Our goal is to determine the optimal way to cut the rod to maximize the total revenue.



Code

```
int cutRod(int arr[], int n) {  
    if(n== 0) return 0;  
    int q = INT_MIN;  
  
    for(int i = 1; i<=n; i++ ) {  
        q = max(q, arr[i] + cutRod(arr, n-i));  
    }  
    return q;  
}
```



Using Memoization

```
int dp[30];
int cutRod(int arr[], int n) {
    if(n== 0) return 0;
    if(dp[n]!=-1) return dp[n];
    int q = INT_MIN;

    for(int i = 1; i<=n; i++ ) {
        q = max(q, arr[i] + cutRod(arr, n-i));
        dp[n] = q;
    }
    return q;
}
```